Addon Development Guide

This guide explains how to build, test, and distribute addons for the Projection Mapper app.

Quick Start

1. Create a folder for your addon (e.g. `my-addon/`)
2. Add an `addon.json` manifest (see below)
3. Create your entry point file (e.g. `index.js`)
4. Export `activate` and `deactivate` functions
5. Install via **Addons → Browse Marketplace** or copy to the addons directory

Addon Manifest (`addon.json`)

Every addon requires an `addon.json` manifest in its root directory:

```
{
  "id": "my-addon",
  "name": "My Addon",
  "version": "1.0.0",
  "description": "A short description of what this addon does",
  "author": "Your Name",
  "entry": "index.js",
  "category": "tool",
  "permissions": ["projection:read", "storage:read", "storage:write"],
  "settings": {
    "myOption": {
      "type": "string",
      "label": "My Option",
      "description": "Describe the setting",
      "default": "default-value"
    }
  },
  "minAppVersion": "0.7.0"
}
```

Required Fields

Field	Type	Description
`id`	`string`	Unique identifier (lowercase, hyphens allowed)
`name`	`string`	Human-readable display name
`version`	`string`	Semver version (e.g. `1.0.0`)
`description`	`string`	Brief description
`author`	`string`	Author name or organisation
`entry`	`string`	Entry point file relative to addon root
`category`	`string`	One of: `effect`, `integration`, `import_export`, `tool`
`permissions`	`string[]`	Required permissions (see below)

Optional Fields

Field	Type	Description
`settings`	`array`	User-configurable settings schema
`minAppVersion`	`string`	Minimum app version required

Permissions

Addons must declare the permissions they need. Users see these before enabling.

Permission	Description
`projection:read`	Read projection surfaces and config
`projection:write`	Modify projection content
`storage:read`	Read addon persistent storage
`storage:write`	Write addon persistent storage
`ui:notification`	Show notifications to user
`network:http`	Make HTTP requests

Addon API

Your addon receives a sandboxed API instance when activated. The API surface depends on the permissions declared in your manifest.

Entry Point

```
// index.js
let api;

async function activate(addonAPI) {
  api = addonAPI;
  api.log.info('My addon is active!');
}

async function deactivate() {
  api.log.info('My addon is shutting down');
  api = null;
}

module.exports = { activate, deactivate };
```

API Reference

`api.events`

Event bus for inter-addon and app communication.

```
// Subscribe to events
const unsub = api.events.on('projection:update', (event) => {
  console.log('Projection changed:', event);
});

// Emit custom events
api.events.emit('my-addon:custom-event', { data: 123 });

// Clean up
unsub();
```

Built-in events:

- `addon:loaded` — Addon was loaded

- `addon:enabled` — Addon was enabled
- `addon:disabled` — Addon was disabled
- `addon:unloaded` — Addon was unloaded
- `addon:error` — Addon encountered an error
- `addon:settings-changed` — Addon settings were modified
- `projection:update` — Projection content changed
- `projection:surface-added` — New surface added
- `projection:surface-removed` — Surface removed

`api.projection` (requires `projection:read`)

```
const surfaces = api.projection.getSurfaces();
// Returns: ProjectionSurface[]
```

`api.storage` (requires `storage:read` / `storage:write`)

```
// Read all stored data for this addon
const data = api.storage.read();

// Write (replaces all stored data)
api.storage.write({ counter: 42, lastRun: Date.now() });
```

`api.log`

Logging with automatic addon-ID prefix.

```
api.log.info('Something happened');
api.log.warn('Watch out');
api.log.error('Something went wrong');
```

Settings

Define user-configurable settings in your manifest. The app renders a settings UI automatically.

Setting Types

Type	Description	Extra Fields
`string`	Text input	—
`number`	Numeric input	—
`boolean`	Checkbox toggle	—
`select`	Dropdown selection	`options: []`

Reading Settings at Runtime

Settings are persisted via the storage API:

```
async function activate(api) {
  const settings = api.storage.read();
  const greeting = settings.greeting || 'Hello!';
  api.log.info(greeting);
}
```

Addon Lifecycle

```
installed → loaded → enabled ⇄ disabled → unloaded
                        ↓
                      error
```

1. **Installed** — Addon files are on disk
2. **Loaded** — Manifest parsed and validated
3. **Enabled** — `activate()` called, addon is running
4. **Disabled** — `deactivate()` called, addon is paused
5. **Error** — Something went wrong during load/activation

Directory Structure

```
my-addon/
├── addon.json       # Manifest (required)
├── index.js         # Entry point (required)
├── lib/             # Additional modules (optional)
└── assets/          # Static assets (optional)
```

Example Addon

See `/example-addons/hello-world/` (../example-addons/hello-world/) for a complete working example that demonstrates:

- Lifecycle hooks (`activate` / `deactivate`)
- Event subscription
- Storage API usage
- Logging
- Settings access

Testing Your Addon

1. Copy your addon folder to the app's addons directory
2. Open the app → Sidebar → Addons
3. Your addon should appear in the installed list
4. Enable it and check the console for log output
5. Open addon details to modify settings

Publishing

To publish your addon to the marketplace:

1. Ensure your `addon.json` is complete and valid
2. Test thoroughly with the latest app version
3. Submit via the marketplace at obitron.abacusai.app (https://obitron.abacusai.app)

Troubleshooting

- **Addon not appearing**: Check that `addon.json` is valid JSON and contains all required fields
- **Permission denied**: Ensure you've declared all needed permissions in the manifest
- **Activation error**: Check the developer console for stack traces. The addon state will show as `error`
- **Settings not saving**: Verify `storage:write` permission is declared